Activity 2.3: Login and Register page routings

- Login page

  /login: Authenticates user credentials and initiates a session upon successful login.

  ```
  @app.get("/login", response_class=HTMLResponse)
  async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
      token = await get_token(request)
      if token:
        user = await get_current_user(request, token)
        if user:
          return RedirectResponse(url="/dashboard",
  status_code=status.HTTP_302_FOUND)
    except Exception:
      pass
   return templates.TemplateResponse("login.html", {"request": request})
  ```

- Register Page

  /register: Handles new user registration by accepting and securely storing user credentials.

  ```
  @app.get("/register", response_class=HTMLResponse)
  async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
      token = await get_token(request)
  ```

```
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard",
status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

- Logout Page

  /logout: Terminates the current user session and redirects to the login page.

```
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
```

```
            except JWTError:
                pass

        # Use RedirectResponse instead of JSONResponse for proper redirection
        response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
response.delete_cookie(key="access_token")
```